

Exercise Sheet 1

Tien Minh

May 16, 2026

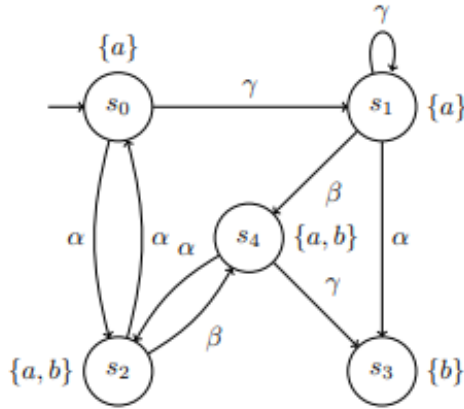
1 Exercise 1

We call a transition system $TS = (S, Act, \rightarrow, S_0, AP, L)$

- action-deterministic if $|S_0| \leq 1$ and $|Post(s, \alpha)| \leq 1$ for all $s \in S$ and $\alpha \in Act$, and
- AP-deterministic if $|S_0| \leq 1$ and $|Post(s) \cap \{s' \in S \mid L(s') = A\}| \leq 1$ for all $s \in S$ and $A \in 2^{AP}$,

where $Post(s, \alpha) = \{s' \in S \mid \exists (s, \alpha, s') \in \rightarrow\}$ and $Post(s) = \cup_{\alpha \in Act} Post(s, \alpha)$

Let the transition system TS_1 be as follows.



- Give the formal definition of TS_1
- Specify a finite and an infinite execution of TS_1

- (c) Decide whether TS_1 is (i) AP-deterministic, and/or (ii) action-deterministic. Justify your answer.

Solution

(a) The formal definition of TS_1 :

- State Space: $S = \{s_0, s_1, s_2, s_3, s_4\}$
- Action: $Act = \{\alpha, \beta, \gamma\}$
- Transition relation:

$$\rightarrow = \{(s_0, \gamma, s_1), (s_0, \alpha, s_2), (s_1, \gamma, s_1), (s_1, \beta, s_4), (s_1, \alpha, s_3), (s_2, \alpha, s_0), \\ (s_2, \beta, s_4), (s_4, \alpha, s_2), (s_4, \gamma, s_3)\}$$

- Initial State: $S_0 = s_0$
- Atomic Propositions: $AP = \{a, b\}$
- Labeling Function:

$$L(s) = \begin{cases} \{a\} & \text{if } s \in \{s_0, s_1\} \\ \{a, b\} & \text{if } s \in \{s_2, s_4\} \\ \{b\} & \text{if } s = s_3 \end{cases}$$

(b) A finite execution of TS_1 :

$$s_0 \xrightarrow{\alpha} s_2 \xrightarrow{\beta} s_4 \xrightarrow{\gamma} s_3$$

An infinite execution of TS_1 :

$$(s_0 \xrightarrow{\alpha} s_2 \xrightarrow{\alpha} s_0)^\omega$$

(c) Based on what has been given, we have:

- $\forall s \in S, \forall \alpha \in Act, \text{if } (|S_0| \leq 1) \wedge (|Post(s, \alpha)| \leq 1) \implies \text{Action-deterministic (1)}$
- $\forall s \in S, A \in 2^{AP}, \text{if } (|S_0| \leq 1) \wedge (|Post(s) \cap \{s' \in S \mid L(s') = A\}| \leq 1) \implies \text{AP-deterministic (2)}$

Because $|S_0| = 1 \implies |S_0| \leq 1 = \text{True (3)}$

- Check TS_1 is AP-deterministic

2^{AP} is the set of all subsets of AP

$$2^{AP} = \{\phi, \{a\}, \{b\}, \{a, b\}\}$$

From (2), (3) $\implies$

For TS_1 is AP-deterministic, $|Post(s) \cap \{s' \in S \mid L(s') = A\}| \leq 1$ must be True

- With $s = s_0$:

$$Post(s_0) = \{s_1, s_2\}$$

- With $A = \phi$

$$\{s' \in S \mid L(s') = A\} = \{\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{\}| = 0 \leq 1 \text{ (satisfied)}$$

- With $A = \{a\}$

$$\{s' \in S \mid L(s') = A\} = \{s_0, s_1\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{s_1\}| = 1 \leq 1 \text{ (satisfied)}$$

- With $A = \{b\}$

$$\{s' \in S \mid L(s') = A\} = \{s_3\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{\}| = 0 \leq 1 \text{ (satisfied)}$$

- With $A = \{a, b\}$

$$\{s' \in S \mid L(s') = A\} = \{s_2, s_4\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{s_2\}| = 1 \leq 1 \text{ (satisfied)}$$

- With $s = s_1$

$$Post(s_1) = \{s_3, s_4\}$$

- With $A = \phi$

$$\{s' \in S \mid L(s') = A\} = \{\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{\}| = 0 \leq 1 \text{ (satisfied)}$$

- With $A = \{a\}$

$$\{s' \in S \mid L(s') = A\} = \{s_0, s_1\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{\}| = 0 \leq 1 \text{ (satisfied)}$$

- With $A = \{b\}$

$$\{s' \in S \mid L(s') = A\} = \{s_3\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{s_3\}| = 1 \leq 1 \text{ (satisfied)}$$
- With $A = \{a, b\}$

$$\{s' \in S \mid L(s') = A\} = \{s_2, s_4\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{s_4\}| = 1 \leq 1 \text{ (satisfied)}$$
- With $s = s_2$

$$Post(s_2) = \{s_0, s_4\}$$
 - With $A = \phi$

$$\{s' \in S \mid L(s') = A\} = \{\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{\}| = 0 \leq 1 \text{ (satisfied)}$$
 - With $A = \{a\}$

$$\{s' \in S \mid L(s') = A\} = \{s_0, s_1\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{s_0\}| = 1 \leq 1 \text{ (satisfied)}$$
 - With $A = \{b\}$

$$\{s' \in S \mid L(s') = A\} = \{s_3\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{\}| = 0 \leq 1 \text{ (satisfied)}$$
 - With $A = \{a, b\}$

$$\{s' \in S \mid L(s') = A\} = \{s_2, s_4\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{s_4\}| = 1 \leq 1 \text{ (satisfied)}$$
- With $s = s_3$

$$Post(s_3) = \{\}$$
 - With $A = \phi$

$$\{s' \in S \mid L(s') = A\} = \{\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{\}| = 0 \leq 1 \text{ (satisfied)}$$
 - With $A = \{a\}$

$$\{s' \in S \mid L(s') = A\} = \{s_0, s_1\}$$

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{\}| = 0 \leq 1 \text{ (satisfied)}$$
 - With $A = \{b\}$

$$\{s' \in S \mid L(s') = A\} = \{s_3\}$$

- $\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{\}| = 0 \leq 1$ (satisfied)
- With $A = \{a, b\}$
 - $\{s' \in S \mid L(s') = A\} = \{s_2, s_4\}$
 - $\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{\}| = 0 \leq 1$ (satisfied)
- With $s = s_4$
 - $Post(s_4) = \{s_2, s_3\}$
 - With $A = \phi$
 - $\{s' \in S \mid L(s') = A\} = \{\}$
 - $\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{\}| = 0 \leq 1$ (satisfied)
 - With $A = \{a\}$
 - $\{s' \in S \mid L(s') = A\} = \{s_0, s_1\}$
 - $\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{\}| = 0 \leq 1$ (satisfied)
 - With $A = \{b\}$
 - $\{s' \in S \mid L(s') = A\} = \{s_3\}$
 - $\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{s_3\}| = 1 \leq 1$ (satisfied)
 - With $A = \{a, b\}$
 - $\{s' \in S \mid L(s') = A\} = \{s_2, s_4\}$
 - $\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| = |\{s_2\}| = 1 \leq 1$ (satisfied)

$$\implies |Post(s) \cap \{s' \in S \mid L(s') = A\}| \leq 1 = \text{True} \quad (4)$$

From (3), (4) $\implies TS_1$ is AP-deterministic

- Check TS_1 is Action-deterministic

From (1) and (3) $\implies$ For TS_1 is action-deterministic, $|Post(s, \alpha)| \leq 1$ must be True

- Case $s = s_0$:
 - $Post(s_0, \gamma) = \{s_1\}, Post(s_0, \alpha) = \{s_2\}$ (satisfied)
- Case $s = s_1$:
 - $Post(s_1, \beta) = \{s_4\}, Post(s_1, \alpha) = \{s_3\}$ (satisfied)
- Case $s = s_2$:
 - $Post(s_2, \beta) = \{s_4\}, Post(s_2, \alpha) = \{s_0\}$ (satisfied)

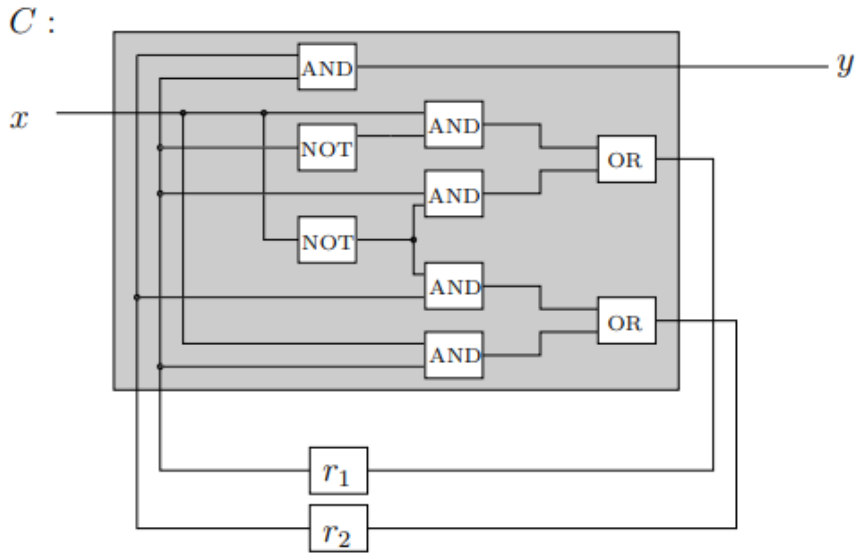
- Case $s = s_3$: (satisfied because there are no way to reach to any state)
- Case $s = s_4$:
 $Post(s_4, \alpha) = \{s_2\}$, $Post(s_4, \gamma) = \{s_3\}$ (satisfied)

$$\Rightarrow |Post(s, \alpha)| \leq 1 = \text{True} \quad (5)$$

From (3), (5) $\Rightarrow TS_1$ is action-deterministic

2 Exercise 2

Consider the following sequential hardware circuit in C.



- Give the function λ_y , δ_{r_1} and δ_{r_2} that govern the changes to the output and register bits depending on the input bits.
- Formally specify the state space of the transition system TS for the circuit C. Justify your answer.
- Draw the transition system TS for the circuit C using the set of labels $AP = \{x, r_1, r_2, y\}$ assuming that initially the values of the registers are 0. Make sure that **all formal components** of the transition system can be uniquely identified from your picture.
- Determine the set $Reach(TS)$.

Solution

(a) Based on the sequential hardware circuit in C, we have these functions:

$$\begin{aligned}\lambda_y &= r_1 \wedge r_2 \\ \delta_{r_1} &= (x \wedge \neg r_1) \vee (r_1 \wedge \neg x) = x \oplus r_1 \\ \delta_{r_2} &= (\neg x \wedge r_2) \vee (x \wedge r_1)\end{aligned}$$

(b) The state space of transition system TS:

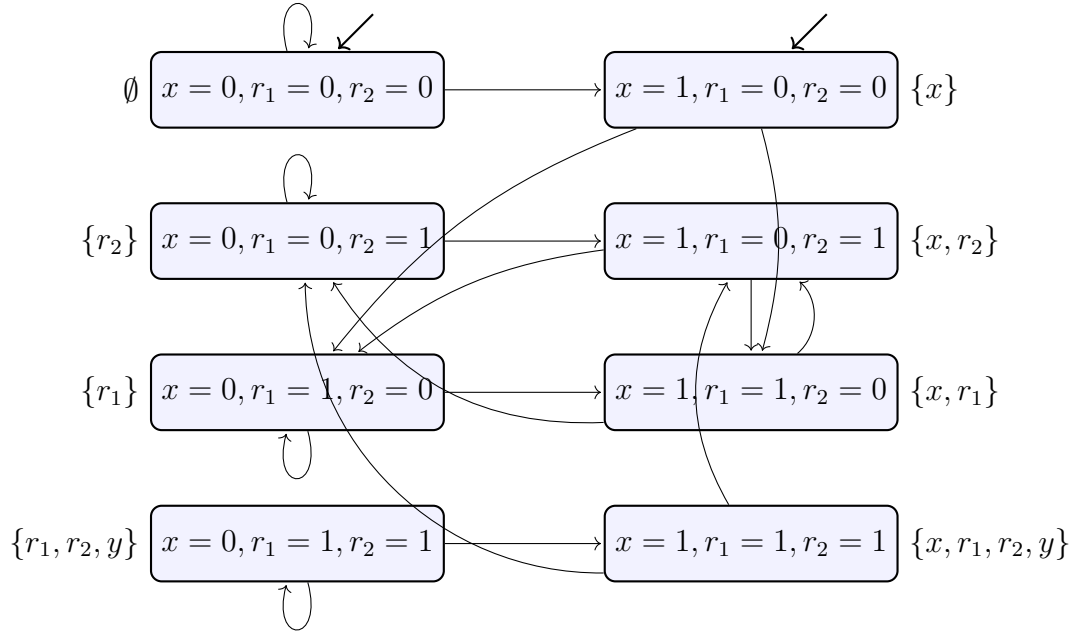
$$\text{State Space: } S = Eval(x) \times Eval(r_1) \times Eval(r_2) \quad (1)$$

$$Eval(x) = Eval(r_1) = Eval(r_2) = \{0, 1\} \quad (2)$$

(1), (2) we conclude:

$$S = \{(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1), (1, 0, 0), (1, 0, 1), (1, 1, 0), (1, 1, 1)\}$$

(c) The transition system of hardware circuit in C as following:



(d) Based on the graph of TS in (c), the $Reach(TS)$ as below:

$$Reach(TS) = \{(0, 0, 0), (1, 0, 0), (0, 0, 1), (1, 0, 1), (0, 1, 0), (1, 1, 0)\}$$

3 Exercise 3*

Consider the following mutual exclusion algorithm for two processes P_0 and P_1 .

The pseudo code of the algorithm for process P_i is given as

```

        int k := 0;
        b := [true, true];
    ℓ0 { while (true) do
        ℓ1 {   b[i] := false;
        ℓ2 {   while (k != i) do
        ℓ2 {       while (not b[1-i]) do
        ℓ2 {           k := i;
        ℓ2 {       end
        ℓ2 {   end
        ℓ3 {   critical_section;
        ℓ4 {   b[i] := true;
        ℓ4 {   end

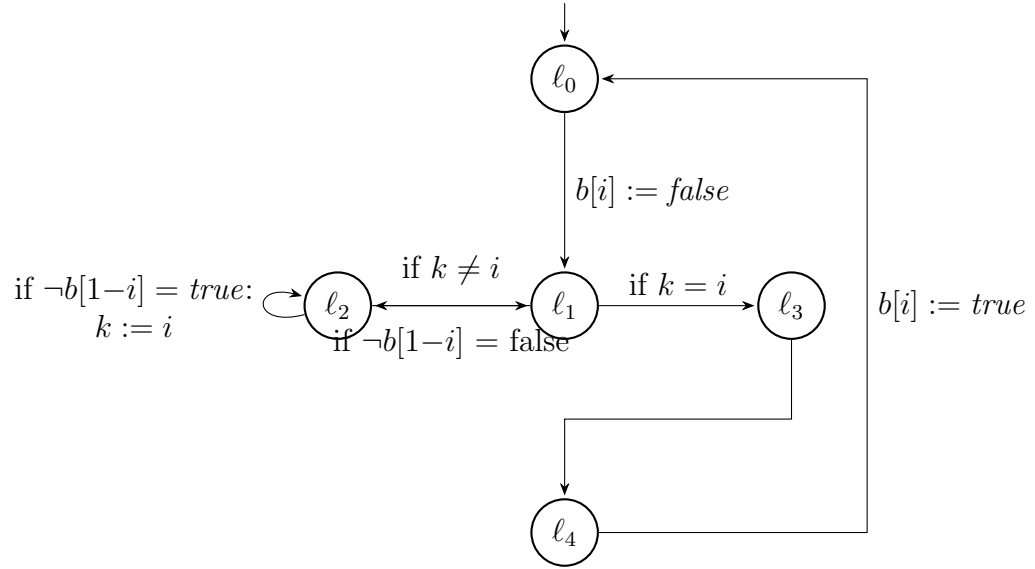
```

where b is an array of two Boolean values which are initially true. k is a variable which is either 0 or 1, and initially 0.

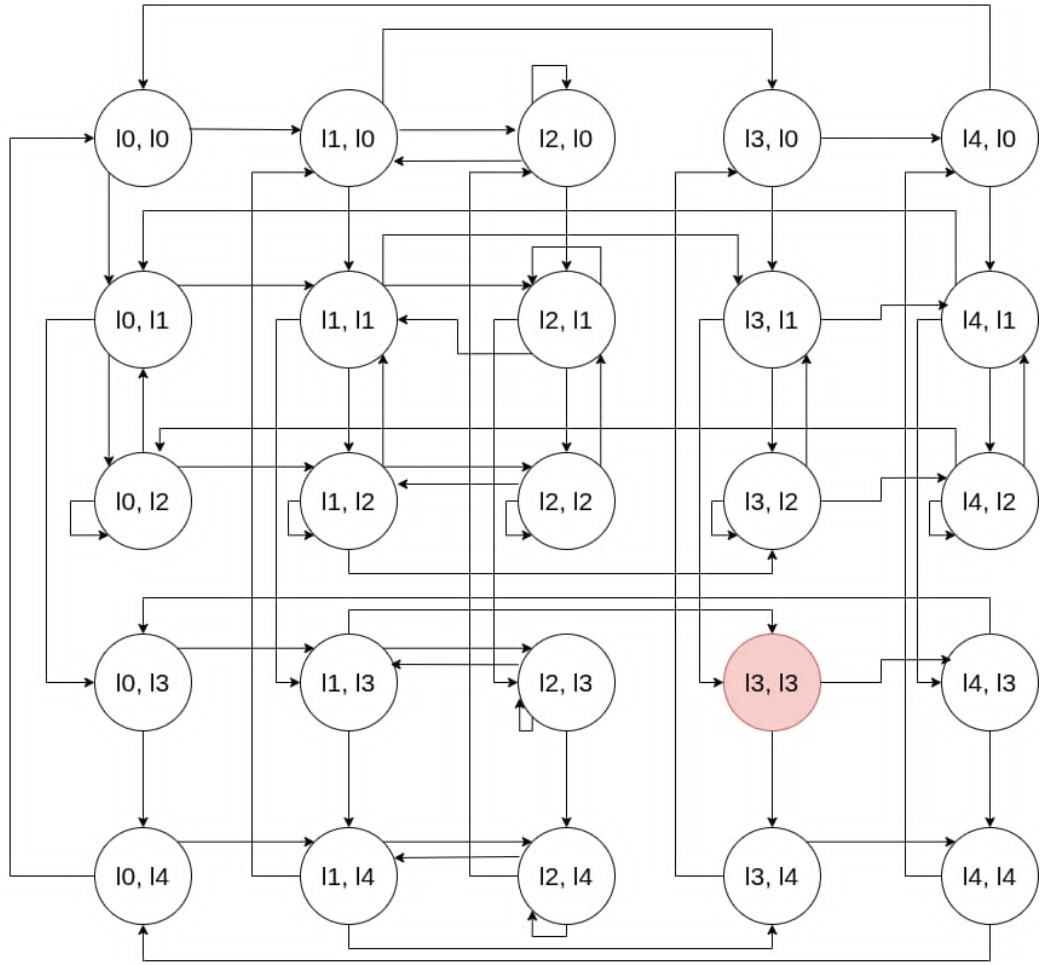
- Give the program graph representation for a single process. Use the locations $\ell_0, \dots, \ell_4$ corresponding to the indicated program fragments.
- Give the reachable part of the transition system of $P_0 || P_1$, i.e., $\text{TS}(P_0 || P_1)$, over the state space $\langle \ell_i, \ell_j, k, b[0], b[1] \rangle$.
Hint: If you wish, you may treat the states $\langle \ell_i, \ell_j, 0, b[0], b[1] \rangle$ and $\langle \ell_j, \ell_i, 1, b[1], b[0] \rangle$ to be equivalent. In other words, whenever you encounter a transition to a state with $k = 1$, you may swap the locations of the processes P_0 and P_1 , swap the values of $b[0]$ and $b[1]$, set k to 0, and make the transition target this “new” state instead. This operation considerably reduces the size of the resulting state space.
- Check whether the algorithm ensures mutual exclusion, i.e., both processes are never in their critical section at the same time. Justify your answer.
- Does the algorithm guarantee that every process eventually enters its critical section? Justify your answer.

Solution

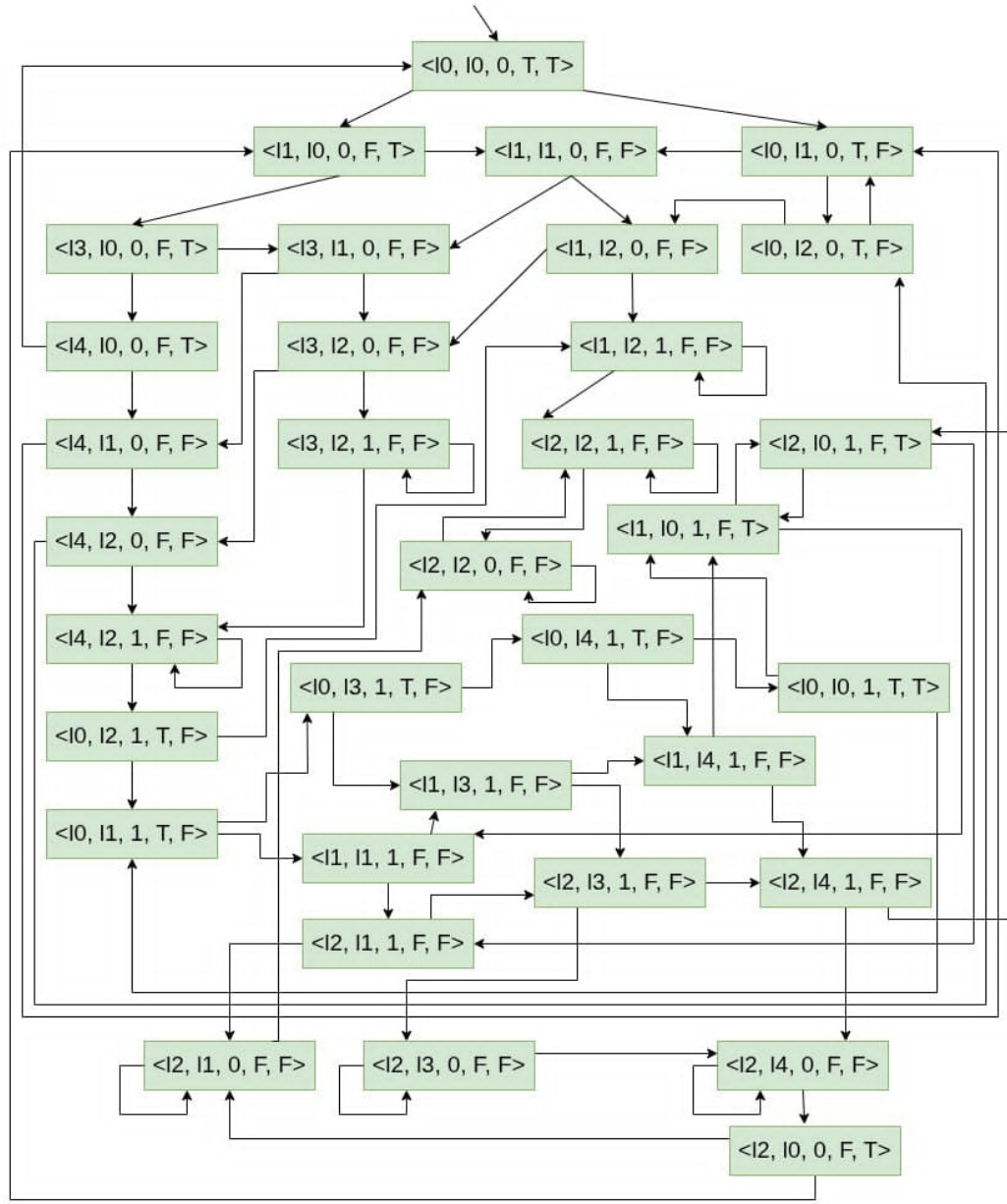
(a) The program graph representation for a single process:



(b) The program graph execution of $P_0 || P_1$ is shown below:



From the program execution, we have the reachable part of TS is shown below:



Note:
- State: $\langle I_i, I_j, k, b[0], b[1] \rangle$
- T = True, F = False

- (c) Let's assume there exists a state where both processes are in the critical section (l_3, l_3) .
Then we have: $b[0] = b[1] = \text{false}$
Besides, $k \in \{0, 1\} \implies$ there exists a process that must run inside the while loop of l_2 .

Because $b[0] = b[1] = \text{false}$, $\implies P_i$ cannot escape the loop to reach l_3 . (contrary to the assumption) Therefore, there does not exist a state where both processes are in the critical section.
 $\implies$ The given algorithm ensures mutual exclusion.

- (d) Based on the reachable state from (b), the algorithm guarantee that every process eventually enters its critical section